Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4.

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Вдосконалення структури коду графічного редактора об’єктів »

Виконала:

студентка групи ІМ-53
Кіосова Софія Олександрівна
Номер в списку - 13

Перевірив:

Рекечинський Д. О

Київ 2026

Завдання

Вимоги

- Усі кольори та стилі (за винятком "гумового" сліду) геометричних форм – як у попередній лабораторній роботі №3.
- "Гумовий" слід при вводі усіх фігур малювати пунктирною лінією.
- Окрім чотирьох типів фігур, які були у попередніх лаб. №2 та 3, запрограмувати ще введення та відображення двох нових фігур – лінія з кружечками та каркас куба.
- Для об’єктів типів лінії з кружечками та каркасу кубу відповідні класи запрограмувати саме множинним успадкуванням.
- Для усіх шести типів форм зробити кнопки Toolbar з підказками (tooltips)

Завдання згідно варіанту

1. Студенти з непарним номером (1, 3, 5, . . .) програмують глобальний статичний об'єкт класу MyEditor

Структура проекту + код

- main.py – головний файл, точка входу у програму

```
import tkinter as tk

from window import Window

from editor import init_global_editor



if __name__ == "__main__":

    root = tk.Tk()

    root.geometry("800x600")

    toolbar_frame = tk.Frame(root, bd=1, relief=tk.RAISED)

    toolbar_frame.pack(side=tk.TOP, fill=tk.X)

    canvas = tk.Canvas(root, bg="lightgray")

    canvas.pack(fill=tk.BOTH, expand=True)

    init_global_editor(canvas)

    app = Window(root, toolbar_frame=toolbar_frame)

    root.mainloop()
```

- editor

```
from shapes import Shape, Point, Line, Rectangle, Ellipse,
line_circles, Cube


class Editor():
    def __init__(self, canvas):
        self.canvas = canvas
        self.MAX_SHAPES = 114
        self.shapes = []
        self.current_shape_type = "Rectangle"


        self.start_x = 0
        self.start_y = 0
        self.rubber_line_id = None
        self.is_drawing = False


        self.canvas.bind('<ButtonPress-1>', self.on_press)
        self.canvas.bind('<B1-Motion>', self.on_drag)
        self.canvas.bind('<ButtonRelease-1>', self.on_release)


    def _clear_rubber_band(self):
        if self.rubber_line_id:
            if isinstance(self.rubber_line_id, (list, tuple)):
                for item_id in self.rubber_line_id:
                    self.canvas.delete(item_id)
            else:
                self.canvas.delete(self.rubber_line_id)
            self.rubber_line_id = None
```

```
    def set_shape_type(self, shape_type):
        self.current_shape_type = shape_type
        self.is_drawing = False
        self.start_x = 0
        self.start_y = 0
        self._clear_rubber_band()

    def on_press(self, event):
        self.is_drawing = True
        self.start_x = event.x
        self.start_y = event.y

    def on_drag(self, event):
        if not self.is_drawing:
            return
        if self.rubber_line_id:
            if isinstance(self.rubber_line_id, (list, tuple)):
                for item_id in self.rubber_line_id:
                    self.canvas.delete(item_id)
            else:
                self.canvas.delete(self.rubber_line_id)
            self.rubber_line_id = None



        #self._clear_rubber_band()
        #temp_shape = None
        #if self.current_shape_type == "Point":
         #   temp_shape = Point(self.start_x, self.start_y, event.x,
event.y)
```

```
        #if temp_shape:
        #    self.rubber_line_id =
temp_shape.draw_rubber_band(self.canvas)
        #if self.rubber_line_id:
        #   self.canvas.delete(self.rubber_line_id)


        temp_shape = None
        if self.current_shape_type == "Point":
            temp_shape = Point(self.start_x, self.start_y, event.x,
event.y)
        elif self.current_shape_type == "Line":
            temp_shape = Line(self.start_x, self.start_y, event.x,
event.y)
        elif self.current_shape_type == "Rectangle":
            temp_shape = Rectangle(self.start_x, self.start_y, event.x,
event.y)
        elif self.current_shape_type == "Ellipse":
            temp_shape = Ellipse(self.start_x, self.start_y, event.x,
event.y)
        elif self.current_shape_type == "line_circles":
            temp_shape = line_circles(self.start_x, self.start_y,
event.x, event.y)
        elif self.current_shape_type == "Cube":
            temp_shape = Cube(self.start_x, self.start_y, event.x,
event.y)


        if temp_shape:
            self.rubber_line_id =
temp_shape.draw_rubber_band(self.canvas)


    def on_release(self, event):
        if not self.is_drawing:
```

```
            return
        self.is_drawing = False
       # self._clear_rubber_band()
        if self.rubber_line_id:
            if isinstance(self.rubber_line_id, (list, tuple)):
                for item_id in self.rubber_line_id:
                    self.canvas.delete(item_id)
            else:
                self.canvas.delete(self.rubber_line_id)
            self.rubber_line_id = None

        shape_obj = None
        if len(self.shapes) >= self.MAX_SHAPES:
            print(f"Досягнуто ліміту об'єктів ({self.MAX_SHAPES})!")
            return

        if self.current_shape_type == "Point":
            shape_obj = Point(self.start_x, self.start_y)
        elif self.current_shape_type == "Line":
            shape_obj = Line(self.start_x, self.start_y, event.x,
event.y)
        elif self.current_shape_type == "Rectangle":
            shape_obj = Rectangle(self.start_x, self.start_y, event.x,
event.y)
        elif self.current_shape_type == "Ellipse":
            shape_obj = Ellipse(self.start_x, self.start_y, event.x,
event.y)
        elif self.current_shape_type == "line_circles":
            shape_obj = line_circles(self.start_x, self.start_y,
event.x, event.y)
```

```
        elif self.current_shape_type == "Cube":
            shape_obj = Cube(self.start_x, self.start_y, event.x,
event.y)

        if shape_obj:
            shape_obj.draw(self.canvas)
            self.shapes.append(shape_obj)
            print(f"Додано об'єкт {self.current_shape_type}. Всього у
масиві: {len(self.shapes)}/{self.MAX_SHAPES}")


_global_editor = None
def init_global_editor(canvas):
    global _global_editor
    if _global_editor is None:
        _global_editor = Editor(canvas)
    return _global_editor
def get_editor():
    return _global_editor
```

- shapes

```
from abc import ABC, abstractmethod


class Shape(ABC):
    def __init__(self, x1, y1, x2=0, y2=0):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2
```

```
    @abstractmethod
    def draw(self, canvas):
        pass

    @abstractmethod
    def draw_rubber_band(self, canvas):
        pass


class Point(Shape):
    def draw(self, canvas):
        return canvas.create_oval(self.x1 - 2, self.y1 - 2, self.x1 +
2, self.y1 + 2, fill="black")
    def draw_rubber_band(self, canvas):
        return canvas.create_oval(self.x1 - 2, self.y1 - 2, self.x1 +
2, self.y1 + 2, fill="red")


class Line(Shape):
    def draw(self, canvas):
        return canvas.create_line(self.x1, self.y1, self.x2, self.y2,
fill="blue")
    def draw_rubber_band(self, canvas):
        return canvas.create_line(self.x1, self.y1, self.x2, self.y2,
fill="red", dash=(4, 2))


class Rectangle(Shape):
    def draw(self, canvas):
        return canvas.create_rectangle(
            self.x1, self.y1,
            self.x2, self.y2,
            outline="black", fill="#8BF7FF"
```

```
        )
    def draw_rubber_band(self, canvas):
        return canvas.create_rectangle(
            self.x1, self.y1,
            self.x2, self.y2,
            outline="red", dash=(4, 2)
        )
"""
class Ellipse(Shape):
    def draw(self, canvas):
        rx = abs(self.x2 - self.x1)
        ry = abs(self.y2 - self.y1)
        left = self.x1 - rx
        top = self.y1 - ry
        right = self.x1 + rx
        bottom = self.y1 + ry
        return canvas.create_oval(
            left, top,
            right, bottom,
            outline="black", fill="#72FF8C"
        )



    def draw_rubber_band(self, canvas):
        rx = abs(self.x2 - self.x1)
        ry = abs(self.y2 - self.y1)
        left = self.x1 - rx
        top = self.y1 - ry
        right = self.x1 + rx
```

```
        bottom = self.y1 + ry
        return canvas.create_oval(
            left, top,
            right, bottom,
            outline="red", dash=(4, 2)
        )
"""


class Ellipse(Shape):
    def draw(self, canvas, rx=None, ry=None, fill="#72FF8C"):
        if rx is None:
            rx = abs(self.x2 - self.x1)
        if ry is None:
            ry = abs(self.y2 - self.y1)
        left = self.x1 - rx
        top = self.y1 - ry
        right = self.x1 + rx
        bottom = self.y1 + ry
        return canvas.create_oval(
            left, top,
            right, bottom,
            outline="black", fill=fill
        )


    def draw_rubber_band(self, canvas):
        rx = abs(self.x2 - self.x1)
        ry = abs(self.y2 - self.y1)
        left = self.x1 - rx
        top = self.y1 - ry
```

```
        right = self.x1 + rx
        bottom = self.y1 + ry
        return canvas.create_oval(
            left, top,
            right, bottom,
            outline="red", dash=(4, 2)
        )


# new figures
class line_circles(Line, Ellipse):
    def draw(self, canvas):
        Line.draw(self, canvas)
        Ellipse.draw(self, canvas, rx=5, ry=5, fill="pink")
        ox, oy = self.x1, self.y1
        self.x1, self.y1 = self.x2, self.y2
        Ellipse.draw(self, canvas, rx=5, ry=5, fill="purple")
        self.x1, self.y1 = ox, oy
    """
        canvas.create_oval(
            self.x1 - 5,
            self.y1 - 5,
            self.x1 + 5,
            self.y1 + 5,
            fill="pink"
        )
        canvas.create_oval(
            self.x2 - 5,
            self.y2 - 5,
            self.x2 + 5,
```

```
            self.y2 + 5,
            fill="purple"
        )
    """
    def draw_rubber_band(self, canvas):
        line_id = Line.draw_rubber_band(self, canvas)
        c1 = canvas.create_oval(
            self.x1 - 5, self.y1 - 5, self.x1 + 5, self.y1 + 5,
            outline="red", dash=(2, 2)
        )
        c2 = canvas.create_oval(
            self.x2 - 5, self.y2 - 5, self.x2 + 5, self.y2 + 5,
            outline="red", dash=(2, 2)
        )
        return (line_id, c1, c2)



class Cube(Line, Point):
    def draw(self, canvas):
        x_min=min(self.x1, self.x2)
        x_max=max(self.x1, self.x2)
        y_min=min(self.y1, self.y2)
        y_max=max(self.y1, self.y2)
        W=x_max - x_min
        H=y_max - y_min
        S= min(W, H)
        delta = S * 0.3
        vertices = [
            (x_min, y_min),
```

```
            (x_min + S, y_min),  # 1: R_UP
            (x_min + S, y_min + S),  # 2: R_DOWN
            (x_min, y_min + S),  # 3: L_DOWN
            (x_min + delta, y_min - delta),  # 4: L_UP_BACK
            (x_min + S + delta, y_min - delta),  # 5: R_UP_BACK
            (x_min + S + delta, y_min + S - delta),  # 6: R_DOWN_BACK
            (x_min + delta, y_min + S - delta),  # 7: L_DOWN_BACK
            ]
        edges = [
            (0, 1),
            (1, 2),
            (2, 3),
            (3, 0),
            (4, 5),
            (5, 6),
            (6, 7),
            (7, 4),
            (0, 4),
            (1, 5),
            (2, 6),
            (3, 7),
            ]

        for start_idx, end_idx in edges:
            p1 = vertices[start_idx]
            p2 = vertices[end_idx]
            self.x1, self.y1 = p1
            self.x2, self.y2 = p2
            Line.draw(self, canvas)
```

```
        for px, py in vertices:
            self.x1, self.y1 = px, py
            Point.draw(self, canvas)


    def draw_rubber_band(self, canvas):
        x_min = min(self.x1, self.x2)
        x_max = max(self.x1, self.x2)
        y_min = min(self.y1, self.y2)
        y_max = max(self.y1, self.y2)
        W = x_max - x_min
        H = y_max - y_min
        S = min(W, H)
        return canvas.create_rectangle(
            x_min, y_min, x_min + S, y_min + S, outline="red", dash=(4,
2)
        )
```

- windows

```
import tkinter as tk
from editor import Editor
import os
from PIL import Image, ImageTk


class ToolTip():
    def __init__(self, widget, text):
        self.widget = widget
        self.text= text
```

```
        self.tip_window = None
        self.widget.bind("<Enter>", self.show_tip)
        self.widget.bind("<Leave>", self.hide_tip)

    def show_tip(self, event=None):
        if self.tip_window or not self.text:
            return
        x, y, _cx, _cy = self.widget.bbox("insert")
        x = x + self.widget.winfo_rootx() + 25
        y = y + self.widget.winfo_rooty() + 20
        self.tip_window = tw = tk.Toplevel(self.widget)
        tw.wm_overrideredirect(True)
        tw.wm_geometry(f"+{x}+{y}")
        label = tk.Label(
            tw, text=self.text, justify=tk.LEFT,
            background="#ffffe0", relief=tk.SOLID, borderwidth=1,
            font=("tahoma", "8", "normal")
        )
        label.pack(ipadx=1)

    def hide_tip(self, event=None):
        if self.tip_window:
            self.tip_window.destroy()
            self.tip_window = None

class Window():
    def __init__(self, root, toolbar_frame=None):
        self.root = root
        self.root.title("OOP_lab3 - [Прямокутник]")
```

```
        root.geometry("600x400")
        #self.toolbar_frame = tk.Frame(root, bd=1, relief=tk.RAISED)
        #self.toolbar_frame.pack(side=tk.TOP, fill=tk.X)
        # canvas = tk.Canvas(root, bg="lightgray")
        # canvas.pack(fill=tk.BOTH, expand=True)
        #self.editor = Editor(canvas)
        #from editor import init_global_editor
        #self.editor = init_global_editor(canvas)
        if toolbar_frame is not None:
            self.toolbar_frame = toolbar_frame
        else:
            self.toolbar_frame = tk.Frame(root, bd=1, relief=tk.RAISED)
            self.toolbar_frame.pack(side=tk.TOP, fill=tk.X)
        from editor import get_editor
        self.editor = get_editor()
        self.icons = {}
        self.buttons = {}
        self.create_menu()
        self.create_toolbar()
        self.select_shape("Rectangle")


    def create_menu(self):
        menubar = tk.Menu(self.root)
        self.root.config(menu=menubar)
        file_menu = tk.Menu(menubar, tearoff=0)
        file_menu.add_command(label="Вихід", command=self.root.quit)
        menubar.add_cascade(label="Файл", menu=file_menu)
        self.objects_menu = tk.Menu(menubar, tearoff=0)
        self.selected_type_var = tk.StringVar(value="Rectangle")
```

```
        shapes = [
            ("Крапка", "Point"),
            ("Лінія", "Line"),
            ("Прямокутник", "Rectangle"),
            ("Еліпс", "Ellipse"),
            ("Лінія з колами", "line_circles"),
            ("Куб", "Cube")
        ]

        for label, shape_type in shapes:
            self.objects_menu.add_radiobutton(
                label=label,
                variable=self.selected_type_var,
                value=shape_type,
                command=lambda st=shape_type: self.select_shape(st)
            )

        menubar.add_cascade(label="Об’єкти", menu=self.objects_menu)
        help_menu = tk.Menu(menubar, tearoff=0)
        help_menu.add_command(label="Про програму",
command=self.show_about)
        menubar.add_cascade(label="Довідка", menu=help_menu)

    def create_toolbar(self):
        base_dir = os.path.dirname(os.path.abspath(__file__))
        tools = [
            ("Point", "Крапка", "dot.png"),
            ("Line", "Лінія", "line.png"),
```

```
            ("Rectangle", "Прямокутник", "rectangle.png"),
            ("Ellipse", "Еліпс", "elipse.png"),
            ("line_circles", "Лінія з колами", "line_circles.png"),
            ("Cube", "Куб", "cube.png")
        ]

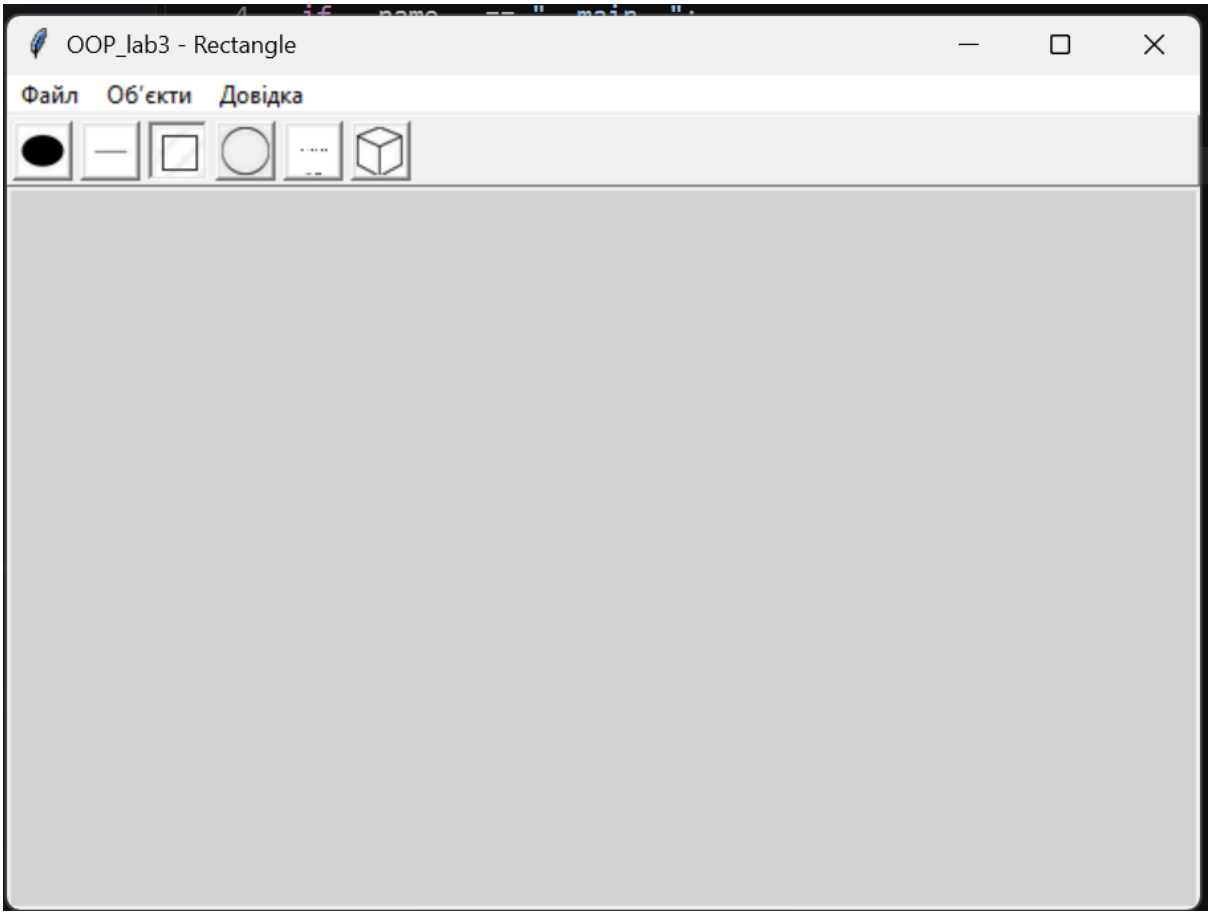
        for shape_type, tooltip_text, icon_file in tools:
            icon_path = os.path.join(base_dir, "images", icon_file)
            try:
                pil_img = Image.open(icon_path).resize((24, 24),
Image.Resampling.LANCZOS)
                img = ImageTk.PhotoImage(pil_img)
                self.icons[shape_type] = img
                # img = tk.PhotoImage(file=icon_path)
                # img = img.subsample(4, 4)
               #  self.icons[shape_type] = img
                btn = tk.Button(
                    self.toolbar_frame, image=img, relief=tk.RAISED,
                    command=lambda st=shape_type: self.select_shape(st)
                )
            except Exception as e:
                print(f"Не вдалося завантажити іконку {icon_path}:
{e}")
                btn = tk.Button(
                    self.toolbar_frame, text=shape_type[0],
relief=tk.RAISED,
                    command=lambda st=shape_type: self.select_shape(st)
                )
            btn.pack(side=tk.LEFT, padx=2, pady=2)
            ToolTip(btn, tooltip_text)
```

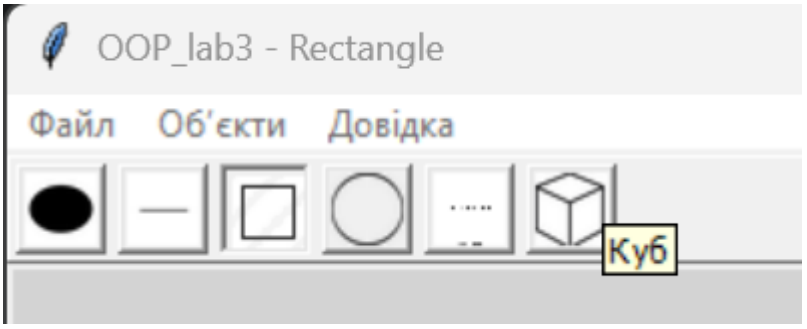
```
            self.buttons[shape_type] = btn

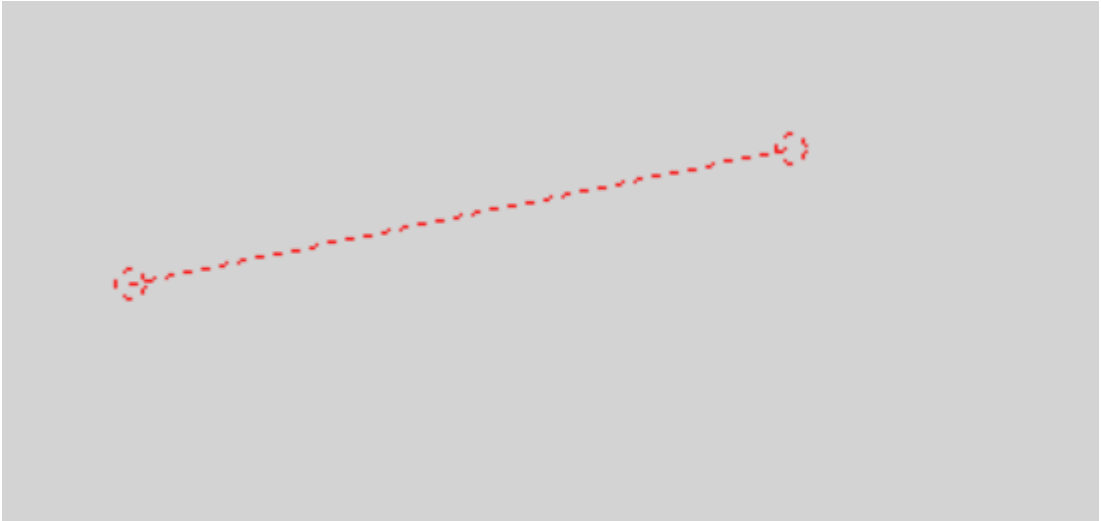

    def select_shape(self, type):
        self.editor.set_shape_type(type)
        self.selected_type_var.set(type)
        for st, btn in self.buttons.items():
            if st == type:
                btn.config(relief=tk.SUNKEN)
            else:
                btn.config(relief=tk.RAISED)
        self.root.title(f"OOP_lab4 - {type}")

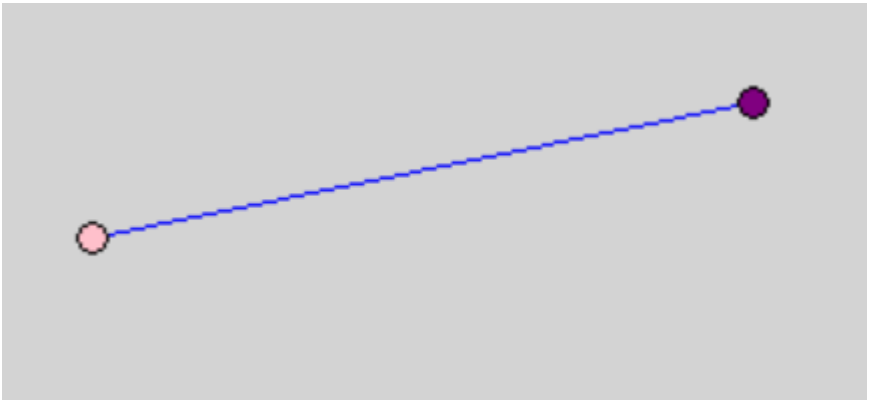

    def show_about(self):
        print("Лабораторна робота №4 з ООП")
```

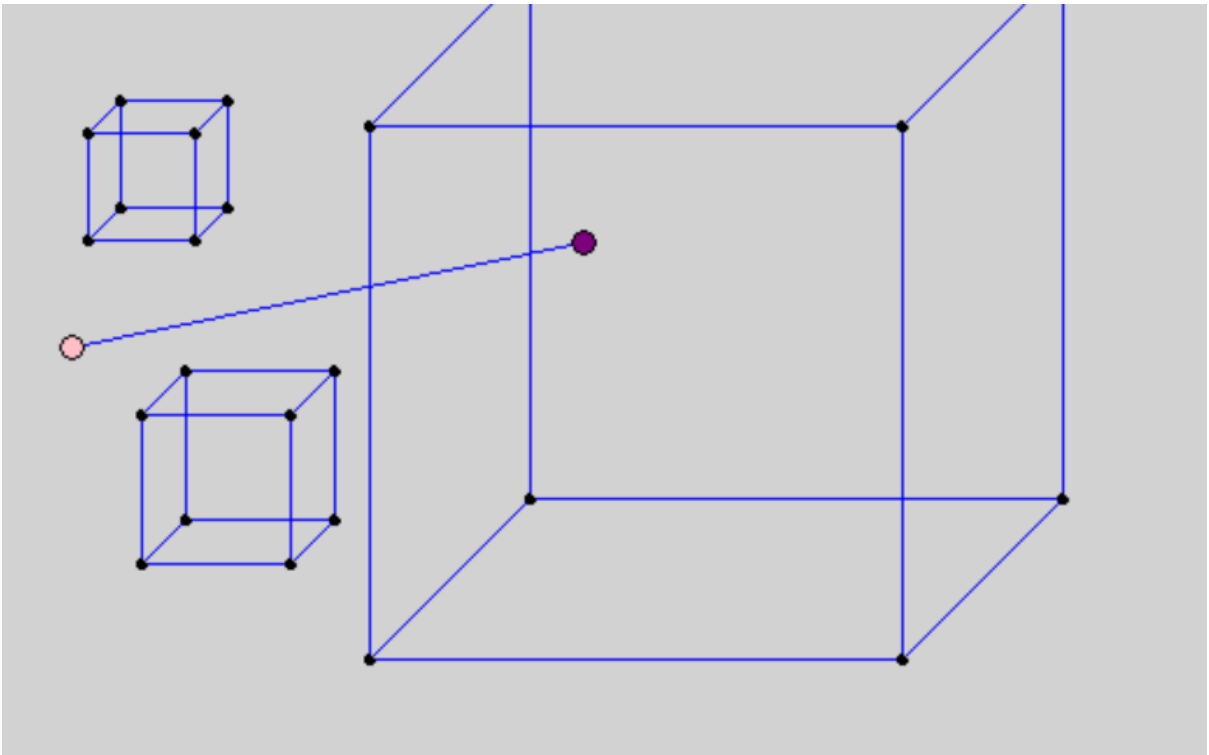
Скріншоти

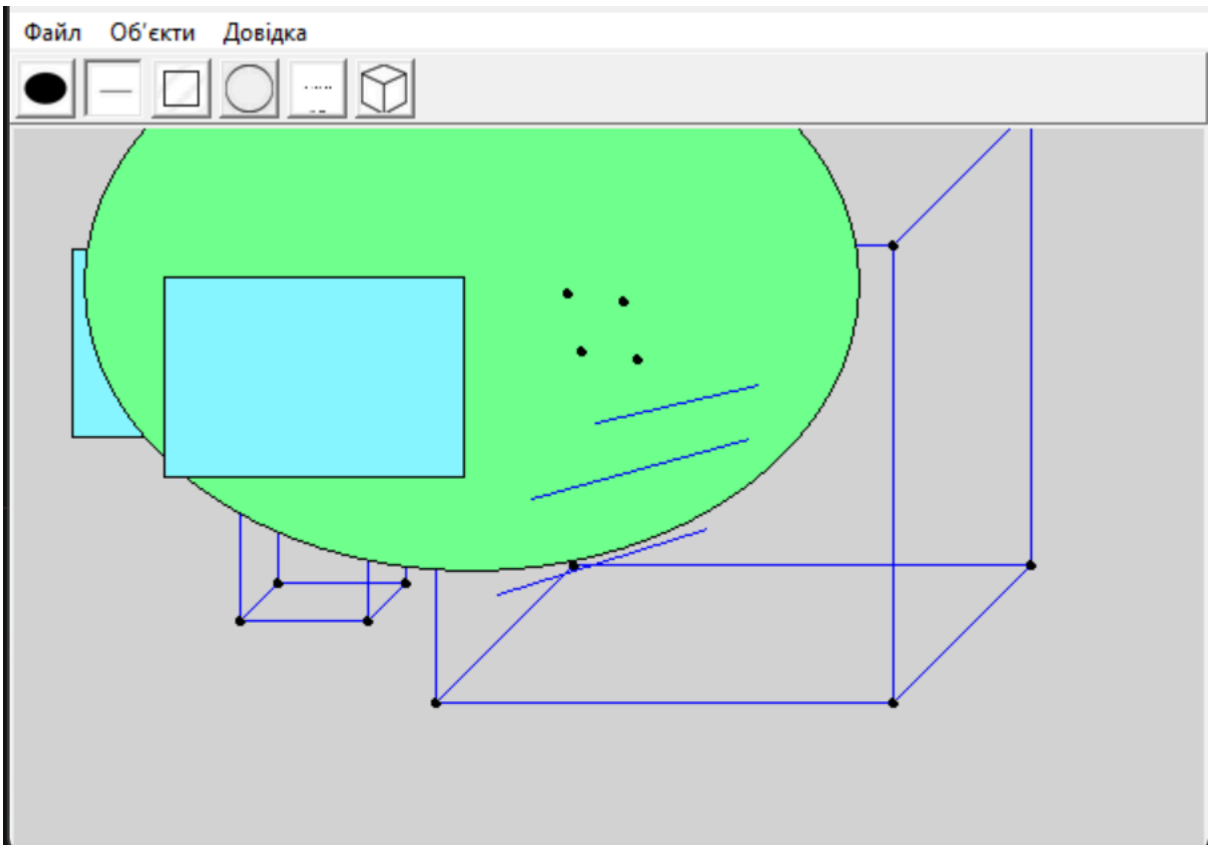

OOP_lab3 - Rectangle
Файл
Об'єкти
Довідка


OOP_lab3 - Rectangle
Файл
Об'єкти
Довідка
Куб

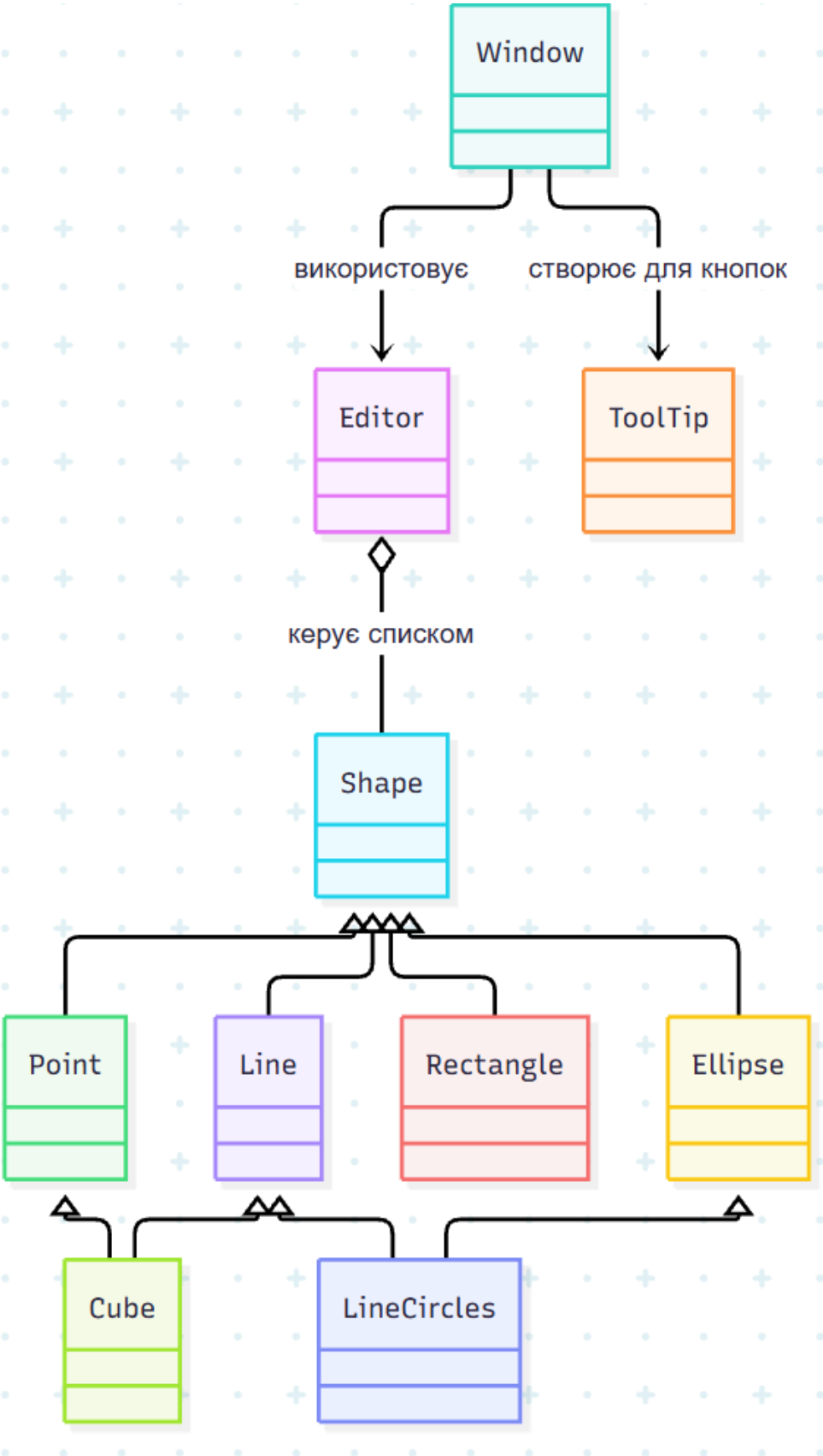







Файл
Об'єкти
Довідка

Діаграма


Window
використовує
створює для кнопок
Editor
ToolTip
керує списком
Shape
Point
Line
Rectangle
Ellipse
Cube
LineCircles

Висновки: Отримала навички програмування глобального статичного об’єкту. Під час виконання практичної роботи розглянула, запрограмувала та зрозуміла різницю у множинному успадкуванні, яке базується на поліморфізмі. Були виправлені помилки, пов’язані із кортежами даних під час множинного успадкування при створенні складних об’єктів: куб та лінія з колом. Було покращенно відображення іконок шляхом застосування бібліотеки *PIL*